Flappy Bird DQN Agent

A Deep Q-Network Reinforcement Learning Project in PyTorch

Vishal Kumar
MCA Student, Jaipur National University
GitHub: https://github.com/vishu56096-ctrl/flappy-bird-dqn-pytorch
LinkedIn: https://www.linkedin.com/in/vishal-kumar-7a4003393/
November 2026

Abstract

This report documents a reinforcement learning project in which a Deep Q-Network (DQN) agent learns to play Flappy Bird without any hand-written rules. The agent is implemented in Python with PyTorch and is trained in the Gymnasium *FlappyBird-v0* environment. The report explains what the project does, why it was built, how it works, the most important parts of the code, the results obtained, the improvements made and planned, and what was learned. It is written so that a reader with no prior knowledge of reinforcement learning can follow it.

Keywords: reinforcement learning, Deep Q-Network, experience replay, target network, epsilon-greedy exploration, PyTorch, Gymnasium.

Contents

1. Introduction 2
2. Purpose and Motivation 2
3. Background Concepts 3
4. System Design 3
5. Key Implementation Details 4
6. Features and Usage 6
7. Training Log and Hyperparameters 7
8. Results and Analysis 7
9. Improvements 8
10. What I Learned 9
11. Repository and Links 9
12. Conclusion 9

1. Introduction

1.1 The project in one sentence

I built an AI agent that teaches itself to play Flappy Bird by trial and error, using a Deep Q-Network written from scratch in PyTorch.

1.2 Plain-language explanation

Flappy Bird has only one control: the bird either flaps or does nothing. The player must keep the bird alive and guide it through gaps between pipes. In this project nobody tells the program how to play. Instead the agent plays thousands of games. Staying alive and passing pipes gives it a reward, and crashing gives it a penalty. By repeating this many times, a neural network gradually learns which action (flap or do nothing) is better in each situation, in the same way a person improves with practice.

This approach is called **reinforcement learning**. It is different from normal machine learning because there is no labelled dataset. The agent creates its own training data by interacting with the game.

1.3 Project at a glance

Table 1: Project summary.

Item	Detail
Project name	Flappy Bird DQN Agent (FlappyBird_RL)
Domain	Reinforcement learning, deep learning
Algorithm	Deep Q-Network (DQN) with experience replay and a target network
Language and libraries	Python, PyTorch, Gymnasium, flappy-bird-gymnasium, PyYAML
Environment	Gymnasium FlappyBird-v0
Control	Command line (agent.py), settings in parameters.yaml
Repository	flappy-bird-dqn-pytorch (GitHub, user vishu56096-ctrl)

2. Purpose and Motivation

2.1 Why I built this project

Reading about reinforcement learning is not the same as making it work. I chose Flappy Bird because the rules are simple, the feedback is immediate, and a failing agent is easy to see. This made it a good first project for understanding how a DQN really behaves, including when it does not work well.

2.2 Objectives

1 Implement a complete DQN training pipeline instead of relying on a ready-made RL library.

2 Understand each building block: Q-values, experience replay, the target network and epsilon-greedy exploration.

3 Build the project like real software: configuration file, command-line options, checkpoints, logging and documentation.

4 Measure the result honestly and identify what limits performance.

5 Publish a clean, reproducible repository with a README and a demo video.

2.3 Scope

The project covers training, resuming, logging and evaluation of one agent in one environment. It does not claim to be a state-of-the-art Flappy Bird solver. It is a learning and portfolio project whose value lies in the implementation, the analysis and the documentation.

3. Background Concepts

The table below explains every key term used in this report in simple words.

Table 2: Key concepts.

Concept	Meaning in plain words	Role in this project
Agent	The learner that makes decisions	The DQN-controlled bird
Environment	The world the agent acts in	The Flappy Bird game (Gymnasium)
State	What the agent currently sees	Numeric observation: bird and pipe information
Action	A choice the agent can make	Flap or do nothing
Reward	Feedback after each action	Positive for surviving and passing pipes, penalty for crashing
Q-value	Predicted total future reward of an action in a state	The network outputs one Q-value per action
DQN	A neural network that estimates Q-values	Policy network and target network
Experience replay	A memory of past moves, sampled at random for training	Replay memory of 100,000 transitions
Target network	A slowly updated copy of the network	Provides stable training targets
Epsilon-greedy	Sometimes act randomly, otherwise act on the best-known action	Epsilon decays from 1.0 to 0.05
Gamma	How much future reward is valued	0.99, so long-term survival matters

For each stored move, the network is trained towards the following target value:

$$y = r + gamma \; x \; (1 - terminated) \; x \; max \; Q_{target}(s', a')$$

Here r is the reward received, s' is the next state, and *terminated* is 1 when the bird crashed. If the game ended, there is no future reward, so the target is just r.

4. System Design

4.1 Project structure

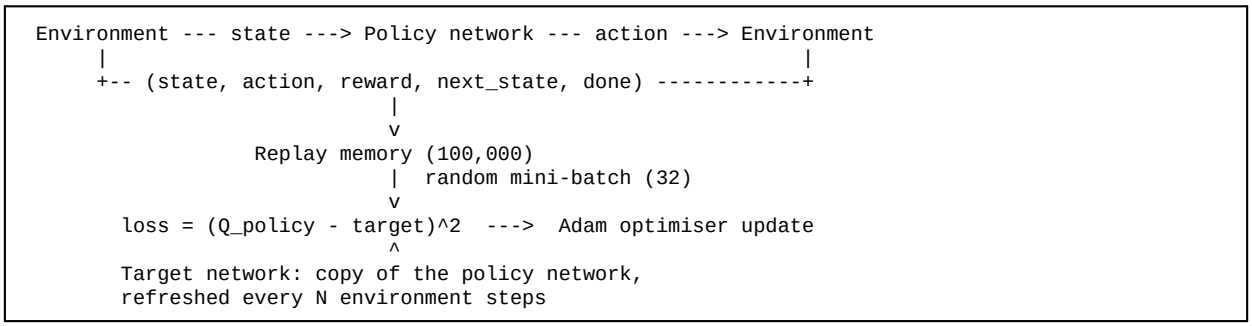
Listing 1: Repository layout.

```
FlappyBird_RL/
|-- agent.py                 training, evaluation, logging, command line
|-- dqn.py                   neural network (hidden layer of 256 units)
|-- experience_replay.py     replay memory
|-- parameters.yaml          all hyperparameters
|-- requirements.txt         dependencies
|-- README.md                documentation
|-- assets/
|   |-- demo.gif             inline README preview
|   `-- Video Project 1.mp4  full gameplay / training demo
`-- runs/                    created when the agent runs
    |-- flappybirdv0.pt              best-reward model weights
    |-- flappybirdv0_latest.pt       latest weights, used for resuming
    |-- flappybirdv0.log             log of new best rewards
    `-- flappybirdv0_episodes.csv    per-episode metrics
```

Training creates and updates the files in *runs/*. The .gitignore excludes temporary files but keeps this agent's checkpoints and training history in Git, so a clone can inspect the CSV and play the best saved model immediately.

4.2 How the pieces work together

Figure 1: Data flow during training.



4.3 Training workflow

1 Reset the game and read the first state.

2 Pick an action: random with probability epsilon, otherwise the action with the highest Q-value.

3 Apply the action and receive the next state, the reward and the end-of-game flag.

4 Store the move in replay memory.

5 Once enough moves are stored, sample a random mini-batch and update the policy network.

6 Every N steps, copy the policy network weights to the target network.

7 At the end of each episode, reduce epsilon, save the latest model, save the best model if the reward is a new record, and append a row to the CSV log.

5. Key Implementation Details

This section shows the most important parts of *agent.py* with a short explanation of each.

5.1 Hyperparameters in a configuration file

Listing 2: parameters.yaml.

```yaml
flappybirdv0:
  env_id: FlappyBird-v0
  epsilon_init: 1
  epsilon_min: 0.05
  epsilon_decay: 0.9995
  replay_memory_size: 100000
  mini_batch_size: 32
  network_sync_rate: 10
  alpha: 0.001
  gamma: 0.99
  reward_threshold: 1000
```

All settings live in one YAML file, selected by name from the command line. Experiments can be changed without editing the Python code, and several parameter sets can exist side by side.

5.2 Epsilon-greedy action selection

Listing 3: Explore or exploit.

```python
if is_training and random.random() < epsilon:
    action = torch.tensor(env.action_space.sample(),
                          dtype=torch.long, device=device)
else:
    with torch.no_grad():
        action = (policy_dqn(state.unsqueeze(dim=0))
                  .squeeze(dim=0).argmax())
```

With probability epsilon the agent tries a random action (exploration). Otherwise it asks the network for the Q-value of each action and takes the best one (exploitation). During evaluation the random branch is switched off.

5.3 Storing experience and training step

Listing 4: Replay memory and target network synchronisation.

```
memory.append((state, action, next_state, reward_tensor, terminated))
steps += 1
if len(memory) >= self.mini_batch_size:
    mini_batch = memory.sample(self.mini_batch_size)
    self.optimize(mini_batch, policy_dqn, target_dqn)

if steps >= self.network_sync_rate:
    target_dqn.load_state_dict(policy_dqn.state_dict())
    steps = 0
```

Every move is saved, and as soon as the memory holds one batch, learning starts. Training on random samples instead of consecutive frames removes the strong correlation between neighbouring moves. The target network is refreshed at a fixed interval to keep the learning target steady.

5.4 The learning update

Listing 5: Computing the loss and updating the network.

```
def optimize(self, mini_batch, policy_dqn, target_dqn):
    states, actions, next_states, rewards, terminations = zip(*mini_batch)
    states = torch.stack(states)
    actions = torch.stack(actions)
    next_states = torch.stack(next_states)
    rewards = torch.stack(rewards)
    terminations = torch.tensor(terminations, dtype=torch.float, device=device)

    with torch.no_grad():
        target_q = rewards + (1 - terminations) * self.gamma * \
                   target_dqn(next_states).max(dim=1)[0]

    current_q = policy_dqn(states).gather(
        dim=1, index=actions.unsqueeze(dim=1)).squeeze(dim=1)
    loss = self.loss_fn(current_q, target_q)
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()
```

The target comes from the frozen target network and is computed without gradients. The current estimate is the policy network's Q-value for the action that was actually taken. The difference between the two is the loss, and Adam adjusts the policy network to reduce it.

5.5 Checkpointing and episode logging

Listing 6: Saving the best model, the latest model and the CSV row.

```
if is_best:
    with open(self.log_file, "a", encoding="utf-8") as file:
        file.write(log_msg + "\n")
    torch.save(policy_dqn.state_dict(), self.model_file)
    best_reward = episode_reward

torch.save(policy_dqn.state_dict(), self.latest_model_file)
writer.writerow([episode_number, episode_reward,
                 epsilon, max(best_reward, episode_reward)])
```

Two model files are kept. The *best* file is the highest-reward model and is used for evaluation. The *latest* file is saved after every episode so training can be stopped and resumed at any time. The CSV file records the whole learning history for later graphs.

6. Features and Usage

6.1 Features

- DQN with a policy network and a target network.
- Experience replay with randomly sampled mini-batches.
- Epsilon-greedy exploration with per-episode decay.
- Resume training from the latest saved weights.
- Best-reward model and latest model saved separately.
- Per-episode CSV history and a text log of new best rewards.
- Hyperparameters loaded from *parameters.yaml*.
- Rendered training, headless training, time-limited training and rendered evaluation.

6.2 Installation

Listing 7: Setting up the environment.

```
python -m venv .venv
.venv\Scripts\Activate.ps1          (Windows PowerShell)
source .venv/bin/activate           (macOS / Linux)
python -m pip install -r requirements.txt
```

The dependencies are *flappy-bird-gymnasium*, *gymnasium*, *PyYAML* and *torch*. Python 3.13.9 was used to verify the project. If PowerShell blocks environment activation, open a new terminal with the environment selected or call the virtual environment's Python directly. Anaconda users can activate the environment where these packages are installed.

6.3 Command-line usage

Table 3: Main commands (run from the project folder).

Command	What it does
python agent.py flappybirdv0	Train with the game window for up to 5 minutes (default)
python agent.py flappybirdv0 --episodes 5000 --no-render	Train headless for up to 5,000 episodes
python agent.py flappybirdv0 --minutes 5 --no-render	Train headless with a 5-minute time limit
python agent.py flappybirdv0 --train --render --minutes 5	Explicitly show the game while training
python agent.py flappybirdv0 --play	Watch the best saved model play 5 episodes
python agent.py flappybirdv0 --play --episodes 20	Evaluate over 20 episodes

Training can be stopped at any time with Ctrl+C. The latest weights are saved, so the next run continues from them. A resumed run starts with a new replay buffer and optimiser, and epsilon is reset to its initial value; this is one of the planned improvements.

7. Training Log and Hyperparameters

7.1 Episode CSV

Table 4: runs/flappybirdv0_episodes.csv columns.

Column	Meaning
episode	Episode number, continued from the existing file when training resumes
reward	Total reward collected in that episode
epsilon	Exploration probability after the episode
best_reward_so_far	Highest reward recorded so far

The file opens directly in Excel or Google Sheets, which makes it easy to plot the learning curve. Evaluation runs do not write to this file. The separate *flappybirdv0.log* records only new best rewards, and the .pt files contain model weights, not episode metrics.

7.2 Hyperparameters used

Table 5: Values from parameters.yaml.

Parameter	Value	Description
Learning rate (alpha)	0.001	Step size of the Adam optimiser
Discount factor (gamma)	0.99	Weight given to future rewards
Replay memory capacity	100,000	Maximum stored transitions
Mini-batch size	32	Transitions per gradient update
Target sync interval	10 steps	Policy weights copied to target network
Initial epsilon	1.0	Start with fully random exploration
Minimum epsilon	0.05	Lowest exploration probability
Epsilon decay	0.9995	Multiplier applied after each episode
Reward threshold	1,000	Safety cap that ends an episode
Hidden layer width	256	Units in the network's hidden layer

8. Results and Analysis

Table 6: Results from the saved files.

Measure	Result
Training episodes recorded	17,346
Best reward during training	9.1
Best model, average over 20 evaluation episodes	2.69
Highest reward in the 20-episode evaluation	6.4
Evaluation setup	No rendering, seeds 2026 to 2045

8.1 Interpretation

The agent clearly learned something: it survives past the start and sometimes clears pipes, which a random player rarely does. However, an average of 2.69 shows that it is not yet a strong player, and the best training reward of 9.1 comes from a single favourable episode. I report these numbers as they are, because honest measurement is more useful than an inflated score.

Two details make the numbers trustworthy. First, evaluation uses 20 episodes with fixed seeds, so the result can be reproduced. Second, the average is reported next to the best single value, so one lucky run cannot hide the typical performance.

8.2 Why performance is limited

Analysing the code and the logs pointed to several likely causes: the target network is synchronised very often, resuming restarts exploration from the maximum, the loss function is sensitive to large errors, and the best model is chosen from one noisy training episode. These are addressed in the next section.

9. Improvements

9.1 Improvements already built into the project

Table 7: Implemented improvements.

Improvement	Benefit
Resume from latest weights	Long training can be paused and continued
Separate best and latest models	Best model stays safe while training continues
Per-episode CSV logging	Learning progress can be graphed and analysed
Time-limited and headless training	Fast experiments without opening a window
YAML configuration	Change experiments without editing code
Fixed-seed evaluation mode	Reproducible, comparable results
Automatic device selection	Works on CPU, CUDA and MPS
Detailed README and demo video	Anyone can install, run and understand the project

9.2 Identified improvements for the next version

Table 8: Planned improvements.

Area	Current	Planned	Reason
Target sync	Every 10 steps	About 100 steps	Target network barely differs from the policy network, so training is less stable
Loss function	MSE	Huber (SmoothL1) with gradient clipping	Less sensitive to large errors
Resume behaviour	Epsilon restarts at 1.0	Start at minimum epsilon	Avoid throwing away learned behaviour with random moves
Batch size	32	64	Smoother gradient estimates
Best-model choice	One training episode	Average of many evaluation episodes	More reliable than a single noisy episode
Evaluation code	-	Define the episode offset outside the training branch	Keep --play free of undefined-variable errors

10. What I Learned

10.1 Technical learning

- How Q-learning extends to neural networks, and why a Q-table cannot scale to continuous states.
- Why experience replay and a target network exist, and what instability appears without them.
- How the exploration-exploitation balance is controlled through epsilon and its decay.
- How to build a PyTorch training loop: tensors, device handling, loss, backpropagation and optimiser steps.
- That hyperparameters such as sync rate, batch size and epsilon schedule can matter as much as the model itself.

10.2 Engineering learning

- Structuring a project into separate files for the agent, the network, the memory and the settings.
- Building a command-line interface with argparse, including validation of conflicting options.
- Making long experiments safe with checkpoints, resume support, a time limit and per-episode CSV logging.
- Evaluating with fixed seeds so results can be repeated.

10.3 Professional learning

- Writing clear documentation: README, setup steps, command table and metric definitions.
- Publishing a project on GitHub with a sensible layout, and reporting results honestly with their limitations.
- Presenting the work with a demo video and a LinkedIn post.

11. Repository and Links

Table 9: Project links.

Resource	Location
GitHub repository	https://github.com/vishu56096-ctrl/flappy-bird-dqn-pytorch
LinkedIn	https://www.linkedin.com/in/vishal-kumar-7a4003393/
Demo GIF (README preview)	assets/demo.gif
Demo video (in repository)	assets/Video Project 1.mp4 (about 26 MB)
Author	Vishal Kumar

Repository contents: source code (agent.py, dqn.py, experience_replay.py), the configuration file (parameters.yaml), requirements.txt, README.md, the demo GIF and video, and the saved models (runs/). Checkpoints, the log and the episode CSV are kept in Git, so a clone can inspect the training history and play the saved agent right away.

12. Conclusion

This project took a reinforcement learning idea from theory to a working, documented program. A DQN agent was implemented from scratch, trained over more than 17,000 episodes and evaluated with reproducible settings. The agent has learned basic survival behaviour but is not yet a strong player. The more valuable outcome is the understanding gained: how each DQN component works, which settings affect stability, and how to run, log and present an experiment professionally. The next step is to apply the planned improvements, retrain, re-evaluate and update the README and this report.